

Network Traffic Analysis with tcpdump

A Practical Guide to DNS, ICMP, and TCP Investigation

Emmanuel Aidoo

2026

Contents

I	Foundations	1
1	Why Network Traffic Analysis Matters	2
1.1	What This Book Covers	2
2	Networking Layers and Protocols at a Glance	3
2.1	Not Everything Rides on TCP or UDP	3
3	Types of Traffic on a Network	5
3.1	By Purpose	5
3.2	By Delivery Pattern	5
II	Getting Started with tcpdump	6
4	Installing tcpdump	7
4.1	On Kali Linux	7
4.2	On Ubuntu	7
5	Finding Your Network Interface	9
5.1	Reading the Output	9
6	Anatomy of a tcpdump Command	10
6.1	Why Quote the Filter	10
6.2	Understanding -nn	11
6.3	Filtering by Protocol and Port	11
6.4	Other Essential Flags	11
III	Case Study	12
7	The “Destination Port Unreachable” Incident	13
7.1	Scenario	13
7.2	The Packet Capture	13
7.3	Reading the Evidence	13
7.4	Conclusion	14
7.5	Recommended Next Step	14

7.6 What This Teaches	14
IV Hands-On Practicals	15
8 Practical 1 — Basic ICMP Capture	16
9 Practical 2 — DNS Query Capture	17
9.1 Why This Is the Core Skill	18
10 Practical 3 — TCP Handshake Capture	19
11 Practical 4 — Baseline and ARP Traffic	21
11.1 What Typically Appears	21
12 Practical 5 — Capturing Traffic to an External Server	22
V Going Further	23
13 Advanced tcpdump Techniques	24
13.1 Filtering by TCP Flags	24
13.2 Rotating Capture Files	24
13.3 Combining with Other Tools	24
14 Why This Matters for a Cybersecurity Career	25
15 Further Practice and Free Resources	26
15.1 Suggested Next Practicals	26
16 Appendix: Command and Flag Reference	27
16.1 Core Commands	27
16.2 Flag Reference	27
16.3 TCP Flag Reference	28

Part I

Foundations

Chapter 1

Why Network Traffic Analysis Matters

Every action a computer takes on a network — loading a webpage, checking email, syncing the system clock, resolving a domain name — produces packets. A packet is the basic unit of data sent across a network: a small envelope of information with headers describing where it came from, where it is going, and what protocol it is using, wrapped around whatever payload it is carrying.

Network traffic analysis is the practice of capturing and reading these packets to answer very practical questions:

Is this machine sending traffic? Where is it connecting? Which protocol or port is being used? Why is this connection failing?

These are the same questions a help-desk technician asks when a user cannot load a website, and the same questions a Security Operations Center analyst asks when investigating a suspicious alert. The tool at the center of this book, `tcpdump`, is one of the oldest and most trusted ways to answer them — a lightweight, command-line packet analyzer available on virtually every Linux system.

1.1 What This Book Covers

This book walks through a real diagnostic scenario — a website reported as unreachable, traced back to a broken DNS server — and then builds out a series of independent, hands-on practicals so the reader can reproduce similar analysis on their own machine, against real traffic they generate themselves.

Read a chapter, then run the commands shown. Packet analysis is a skill built by watching real traffic, not by reading about it.

Chapter 2

Networking Layers and Protocols at a Glance

Networking is commonly described using the seven-layer OSI model. Each layer has a distinct job, and different protocols operate at different layers. Understanding this is essential to reading packet captures, because it explains why some protocols, such as ICMP, do not use ports, while others, such as DNS, do.

Layer	Name	Job	Example Protocols
7	Application	User-facing services	HTTP, HTTPS, DNS, FTP, SMTP, IMAP, SSH, NTP, DHCP
6	Presentation	Data formatting, encryption	TLS/SSL, MIME
5	Session	Managing connections between apps	NetBIOS, RPC
4	Transport	End-to-end delivery and reliability	TCP, UDP
3	Network	Logical addressing and routing	IP, ICMP, ICMPv6, IPsec
2	Data Link	Node-to-node delivery on a local network	Ethernet, Wi-Fi, ARP, PPP
1	Physical	Raw transmission of bits	Cabling, radio waves, fiber, DSL

2.1 Not Everything Rides on TCP or UDP

A common misconception is that all network protocols are carried inside TCP or UDP. Most application-layer protocols are — but two protocols that appear constantly in captures are not.

ICMP operates directly on top of IP, at Layer 3. It has no port number, which is why an ICMP error message about a UDP port problem is still not itself a UDP packet.

ARP operates below IP entirely, at Layer 2. It does not use IP addressing for its own delivery; it works directly with hardware, or MAC, addresses on the local network segment.

Why this matters in practice. When a capture is filtered with `udp port 53`, the ICMP error responses related to a failed DNS lookup will not appear, because ICMP is not UDP. This is why diagnostic filters often combine protocols explicitly, for example `udp port 53 or icmp`.

Chapter 3

Types of Traffic on a Network

Beyond the OSI layers, it is useful to categorize traffic by its purpose and its delivery pattern.

3.1 By Purpose

Category	Description	Examples
Application traffic	Tied to a specific user-facing service	Web browsing, email, file transfer, remote login
Transport traffic	The delivery mechanism beneath application traffic	TCP (reliable), UDP (fast, no guarantee)
Control and management traffic	Keeps the network running, not user data	ICMP, ARP, DHCP
Real-time and streaming traffic	Continuous, time-sensitive flows	Video/audio streaming, VoIP, time sync (NTP)

3.2 By Delivery Pattern

Type	Meaning
Unicast	Sent from one device directly to one other device — most everyday traffic
Broadcast	Sent to every device on the local network, for example an ARP request
Multicast	Sent to a specific group of interested devices, for example IPv6 router solicitation

Every capture example in Parts Three and Four of this book maps onto this chapter. DNS and NTP are application traffic riding on UDP; ICMP and ARP are control traffic; a TCP handshake is transport-layer traffic setting up a reliable connection.

Part II

Getting Started with tcpdump

Chapter 4

Installing tcpdump

tcpdump ships pre-installed on most penetration-testing distributions, including Kali Linux. On a general-purpose distribution such as Ubuntu, it usually needs a quick manual install.

4.1 On Kali Linux

Confirm it is already present:

```
tcpdump --version
```

If it is missing for any reason:

```
sudo apt update && sudo apt install tcpdump
```

4.2 On Ubuntu

Step 1 — Update package lists

```
sudo apt update
```

Step 2 — Install tcpdump

```
sudo apt install tcpdump -y
```

Step 3 — Verify the installation

```
tcpdump --version
```

Expected output looks like this:

```
tcpdump version 4.99.6
libpcap version 1.10.6 (64-bit time_t, with TPACKET_V3)
OpenSSL 3.5.5
64-bit build, 64-bit time_t
```

Setting up a lab pair. For several practicals in this book, it helps to have a second machine or virtual machine with no DNS service running — a clean stand-in for a “broken” server.

A plain, freshly installed Ubuntu virtual machine works well for this by default, since it has no local DNS server active out of the box.

Chapter 5

Finding Your Network Interface

Before anything can be captured, tcpdump needs to know which network interface to listen on. The `ip a` command, short for “ip address,” lists every interface on the machine.

```
ip a
```

5.1 Reading the Output

Element	Meaning
Interface number and name	1: <code>lo</code> is the loopback interface, always present. 2: <code>eth0</code> (or <code>enp0s3</code> , <code>wlan0</code>) is a real network card.
Flags in angle brackets	UP means the interface is active; DOWN means it is not currently connected.
inet line	The IPv4 address assigned to that interface, for example <code>172.30.179.167/20</code> .
inet6 line	The IPv6 address, if assigned.
link/ether line	The interface’s MAC, or hardware, address.

The interface to use for capturing is the one marked `state UP` with an `inet` address in the normal range — not `lo`. That name, commonly `eth0`, is what gets passed to tcpdump’s `-i` flag.

Chapter 6

Anatomy of a tcpdump Command

A typical tcpdump command looks like this:

```
sudo tcpdump -i eth0 'udp port 53 or icmp' -nn
```

Every piece has a specific job.

Component	Meaning
sudo	Runs the command with elevated privileges. Capturing raw packets requires the CAP_NET_RAW capability, which normal users do not have by default.
tcpdump	The packet-capture program itself.
-i eth0	Specifies which network interface to listen on. -i any captures on all interfaces at once.
'udp port 53 or icmp'	The capture filter, written in Berkeley Packet Filter syntax. Only packets matching this condition are shown.
-nn	Disables name resolution — IP addresses and ports are shown as raw numbers instead of resolved hostnames or service names.

6.1 Why Quote the Filter

A filter with no spaces or special characters, such as `port 53`, does not strictly need quotes — the shell splits it into separate words, and tcpdump reassembles them internally regardless. Once a filter includes logical operators such as `or` and `and`, or symbols such as parentheses, quoting becomes important: it guarantees the shell treats the whole expression as one argument, rather than risking misinterpretation of special characters. Quoting by default is the safer habit.

6.2 Understanding -nn

This flag is two stacked `-n` flags. The first disables converting IP addresses into hostnames, avoiding reverse DNS lookups. The second disables converting port numbers into service names, so 53 is shown instead of domain.

A subtle trap. Without `-nn`, `tcpdump` performs a reverse DNS lookup on every IP address it displays. When capturing DNS traffic itself, this can generate extra DNS queries that show up inside the very capture being studied. Using `-nn` for DNS analysis avoids this entirely.

6.3 Filtering by Protocol and Port

Filter	Matches
<code>icmp</code>	All ICMP traffic
<code>udp port 53</code>	DNS traffic specifically over UDP
<code>port 53</code>	Port 53 traffic over either UDP or TCP
<code>host 8.8.8.8</code>	Any traffic to or from a specific IP address
<code>net 192.168.1.0/24</code>	Any traffic within a subnet
<code>src port 443 / dst port 443</code>	Traffic filtered by source-only or destination-only

6.4 Other Essential Flags

Flag	Purpose
<code>-X</code>	Print each packet's payload in hex and ASCII, allowing plaintext content to be read directly.
<code>-x</code>	Hex only, no ASCII column.
<code>-w file.pcap</code>	Write raw packets to a file instead of printing them.
<code>-r file.pcap</code>	Read back a previously saved capture file as text.
<code>-c 10</code>	Stop automatically after capturing ten packets.
<code>-A</code>	Print payload as plain ASCII text only.
<code>-e</code>	Include Ethernet and MAC header information in the output.

Part III

Case Study

Chapter 7

The “Destination Port Unreachable” Incident

7.1 Scenario

A cybersecurity analyst works for a company that provides IT services to clients. Customers of one client report they cannot access `www.yummyrecipesforme.com`, seeing the error “destination port unreachable” after the page fails to load. Attempting to reproduce the issue produces the same error. The analyst loads `tcpdump` and tries again while capturing traffic.

Background: the browser first sends a DNS query over UDP to a DNS server to resolve the domain name to an IP address. Only once it has that IP address does it send an HTTPS request to the web server. A failure early in this chain blocks everything downstream.

7.2 The Packet Capture

```
13:24:32.192571 IP 192.51.100.15.52444 > 203.0.113.2.domain: 35084+ A?  
yummyrecipesforme.com. (24)  
13:24:36.098564 IP 203.0.113.2 > 192.51.100.15: ICMP 203.0.113.2  
udp port 53 unreachable, length 254  
  
13:26:32.192571 IP 192.51.100.15.52444 > 203.0.113.2.domain: 35084+ A?  
yummyrecipesforme.com. (24)  
13:27:15.934126 IP 203.0.113.2 > 192.51.100.15: ICMP 203.0.113.2  
udp port 53 unreachable, length 320  
  
13:28:32.192571 IP 192.51.100.15.52444 > 203.0.113.2.domain: 35084+ A?  
yummyrecipesforme.com. (24)  
13:28:50.022967 IP 203.0.113.2 > 192.51.100.15: ICMP 203.0.113.2  
udp port 53 unreachable, length 150
```

7.3 Reading the Evidence

Field	Value	Meaning
Source	192.51.100.15, port 52444	The client sending the DNS query
Destination	203.0.113.2, .domain = port 53	The DNS server
Query type	A?	Requesting the IPv4 address for the domain
Response	ICMP, udp port 53 unreachable	Sent back from the DNS server itself

Three separate query attempts appear roughly two minutes apart — normal client retry behavior after receiving no usable answer.

The key insight. An ICMP “port unreachable” message is generated by a host when a packet arrives at a port with no service listening on it. This message came from 203.0.113.2 itself, for udp port 53, the standard DNS port. That indicates the DNS server host is reachable on the network, but nothing is listening on UDP port 53 there: the DNS service is down, misconfigured, or not running.

7.4 Conclusion

The affected protocol is DNS. The web server and HTTPS were never involved — the browser never got far enough to attempt an HTTPS request, because it could not resolve the domain name in the first place. The DNS server host is up on the network, but its DNS service is not answering queries on port 53.

7.5 Recommended Next Step

Escalate to the team responsible for the DNS server at 203.0.113.2 to check whether the DNS service, such as BIND or named, is running, correctly configured, and bound to port 53, rather than treating this as a firewall or web-server issue.

7.6 What This Teaches

- How to read tcpdump output for DNS (.domain) and ICMP traffic.
- What an ICMP “port unreachable” message indicates about the destination host, not just the network path.
- How to trace a user-facing symptom back to the specific protocol and layer responsible, rather than assuming the web server is at fault.

Part IV

Hands-On Practicals

Chapter 8

Practical 1 — Basic ICMP Capture

The simplest possible capture: watching a ping in action, one request and one reply at a time.

Step 1 — Start the capture, in a first terminal

```
sudo tcpdump -i eth0 icmp -nn
```

Step 2 — Generate traffic, in a second terminal

```
ping 8.8.8.8
```

Let it run for four or five pings, then stop with Ctrl+C.

Step 3 — Read the pairs

Alternating ICMP echo request (the local machine to 8.8.8.8) and ICMP echo reply (8.8.8.8 back to the local machine) lines should appear, roughly one pair per second.

Step 4 (optional) — Try a failing ping

```
ping -c 4 10.255.255.1
```

Try an address unlikely to respond. Compare what shows up — either nothing at all, or a “Destination Host Unreachable” message, depending on where the failure occurs.

Chapter 9

Practical 2 — DNS Query Capture

This practical rebuilds the case study in Chapter Seven using traffic generated directly, and adds two more DNS outcomes to compare against a successful lookup.

Step 1 — Start the capture, in a first terminal

```
sudo tcpdump -i eth0 udp port 53 -nn
```

Step 2 — Run a normal lookup, in a second terminal

```
dig example.com
```

`nslookup example.com` works as well, if `dig` is not installed; install with `sudo apt install dnsutils`.

Step 3 — Try a few different domains

```
dig wikipedia.org
dig github.com
```

Step 4 — Try a domain that does not exist

```
dig thisdomaindoesnotexist7788.com
```

Instead of an A-record answer, an NXDOMAIN, or non-existent domain, response appears — a different DNS outcome worth contrasting against a successful lookup.

Step 5 — Query a specific DNS server directly

```
dig @1.1.1.1 www.google.com
```

The `@1.1.1.1` syntax tells `dig` to bypass the system's default resolver and query Cloudflare's public DNS server directly. In the capture, the destination IP reads 1.1.1.1 instead of the usual resolver. This is the diagnostic move a real analyst makes: manually pointing a query at a known-good server to determine whether a problem is client-side, domain-side, or specific to one DNS server.

Where DNS servers actually live. A DNS server is a separate machine somewhere on the internet or local network — never something installed on a personal computer. 1.1.1.1 runs on Cloudflare's infrastructure; 8.8.8.8 runs on Google's. An operating system may run a

small local caching stub, visible with `cat /etc/resolv.conf`, often showing 127.0.0.53, but it still forwards fresh queries to a real external server whenever the cache has no answer.

9.1 Why This Is the Core Skill

DNS resolution is the first step of nearly every network interaction. That makes it both the most common source of real troubleshooting tickets and one of the most abused protocols in actual attacks: DNS tunneling, where attackers smuggle stolen data out disguised as DNS queries; DNS spoofing or cache poisoning, where a device is fed a fake IP address for a real domain, redirecting traffic to a malicious server; and command-and-control beaconing, where malware “phones home” via DNS lookups to attacker-controlled domains. Knowing what normal DNS traffic looks like is what makes abnormal DNS traffic recognizable later.

Chapter 10

Practical 3 — TCP Handshake Capture

This practical shows the TCP connection setup and teardown, and contrasts readable plaintext over HTTP against encrypted content over HTTPS.

Step 1 — Capture HTTP, in a first terminal

```
sudo tcpdump -i eth0 port 80 -nn -X
```

The `-X` flag, capital X, prints each packet’s payload in hex and ASCII, allowing the actual content to be read, not just headers.

Step 2 — Generate HTTP traffic, in a second terminal

```
curl http://neverssl.com
```

Most sites now redirect automatically to HTTPS; `neverssl.com` is built specifically to stay on plain HTTP for testing purposes such as this.

Step 3 — Identify the handshake

Three packets appear before any data flows:

Flag shown	Meaning
[S]	SYN — “let us connect”
[S.]	SYN-ACK — “agreed”
[.]	ACK — “confirmed”

After that, the HTTP GET request and HTML response follow, readable in plain text thanks to `-X`.

Step 4 — Repeat for HTTPS

```
sudo tcpdump -i eth0 port 443 -nn -X  
curl https://example.com
```

The same SYN, SYN-ACK, ACK handshake appears — TCP itself does not care about encryption. Once data starts flowing, however, the payload shows scrambled, unreadable bytes instead of plaintext: TLS encryption at work.

Step 5 — Notice the teardown

At the end of the exchange, [F.] packets appear — FIN-ACK. FIN means “no more data to send”; combined with an ACK, it forms the polite, formal close of a TCP connection, typically followed by a final ACK from the other side.

TCP versus UDP, side by side. TCP is connection-oriented: it has an explicit open, the SYN/SYN-ACK/ACK sequence, and an explicit close, the FIN-ACK/ACK sequence. UDP, the protocol DNS uses, is connectionless: it has neither. This is exactly why the DNS capture in Chapter Seven showed bare query and response pairs with no handshake at all.

Chapter 11

Practical 4 — Baseline and ARP Traffic

Not all useful captures are triggered on purpose. Running an unfiltered capture for a few minutes reveals what a machine talks to automatically, without any browser tabs open or manual commands run.

```
sudo tcpdump -i eth0
```

11.1 What Typically Appears

Traffic	What It Is
NTP, Network Time Protocol	The operating system periodically syncing its clock against a time server pool.
ICMPv6 Router Solicitation	The machine asking the local network whether an IPv6 router is present, part of automatic IPv6 configuration.
ARP	Local devices resolving IP addresses to MAC addresses, such as a router and machine mutually reconfirming each other after a period of inactivity.

This is precisely why analysts run unfiltered captures on unfamiliar machines: to establish a baseline of normal background traffic, so anything abnormal — an unexpected external IP, an odd port, traffic to a domain that makes no sense for that device — stands out by comparison.

Common background traffic sources include time synchronization, which runs on a timer automatically; operating system and software updates, which check periodically for new packages or patches; DNS pre-fetching and connectivity checks, where the system checks whether it actually has internet access; and background applications such as cloud sync clients or messaging apps polling their servers on their own schedule.

Chapter 12

Practical 5 — Capturing Traffic to an External Server

Step 1 — Pick a target, such as `google.com` or `8.8.8.8`.

Step 2 — Start the capture with a host filter

```
sudo tcpdump -i eth0 host google.com -w external_capture.pcap
```

The host filter matches traffic where that host is either source or destination. The `-w` flag saves the raw packets to a file for later review.

Step 3 — Generate traffic to the target

```
curl https://google.com
```

Step 4 — Stop and read back the capture

```
tcpdump -r external_capture.pcap -nn
```

Step 5 — Analyze the flow

Look for the shape of the conversation: a DNS lookup first, if a domain name was used, then the TCP handshake, then the actual data exchange, then the FIN/ACK teardown.

The same `.pcap` file can also be opened in Wireshark, with `wireshark external_capture.pcap`, for a visual, color-coded breakdown of each packet. `tcpdump` and Wireshark read the same file format, making this a natural next step after a terminal-based capture.

Part V

Going Further

Chapter 13

Advanced tcpdump Techniques

13.1 Filtering by TCP Flags

Specific TCP flags can be isolated directly, rather than watched for by eye:

```
sudo tcpdump -i eth0 'tcp[tcpflags] & tcp-syn != 0'
```

This captures only SYN packets — connection attempts. A burst of SYN packets sent to many different ports in a short time, with no completed handshakes, is a classic signature of a port scan.

Only run scanning or probing techniques such as this against machines and networks that are owned or explicitly authorized for testing — a personal lab virtual machine is the right place to practice this.

13.2 Rotating Capture Files

```
sudo tcpdump -i eth0 -w rotate.pcap -G 60 -W 5
```

This creates a new file every sixty seconds and keeps only the last five, useful for longer-running background monitoring without one enormous file.

13.3 Combining with Other Tools

```
tcpdump -i eth0 -c 20 | grep example.com
```

tcpdump's text output pipes cleanly into standard command-line tools such as `grep` for live filtering, or into `tshark` for scripted, automated analysis of saved capture files.

Chapter 14

Why This Matters for a Cybersecurity Career

Packet-level traffic analysis is a foundational skill across multiple entry points into the cybersecurity field.

Role	How This Skill Applies
SOC or Security Analyst	Investigating DNS-based and other traffic alerts is routine, day-to-day work.
Network or IT Support	Diagnosing connectivity issues, exactly as in the Chapter Seven case study.
Incident Response	Tracing malware communication patterns and command-and-control beaconing.
Penetration Testing	DNS enumeration and traffic analysis are common early reconnaissance steps.

Working through this book builds practice in identifying normal DNS transactions, spotting failures such as NXDOMAIN and port-unreachable errors, redirecting a query to a specific server for diagnosis, reading a TCP handshake and teardown, contrasting encrypted against unencrypted payloads, and establishing a traffic baseline to make abnormal activity stand out. These are the building blocks analysts use when investigating real incidents involving DNS, TCP, and reconnaissance activity such as port scanning.

Chapter 15

Further Practice and Free Resources

To continue building on these fundamentals with more structured, guided material:

Resource	What It Offers
TCP/IP Essentials — tcpdump and Wireshark Lab (GitHub)	Free, open-source lab material covering ICMP capture, saving to file, and cross-referencing with Wireshark.
TCPDump Tutorial — Ethical Hacking Labs (GitHub)	A free tutorial covering flags, filters, and annotated packet output.
Cybrary — TCPDump Virtual Lab	A guided virtual lab environment, free account required, walking through tcpdump options step by step.
Hack The Box — Introduction to Network Traffic Analysis	Scenario-based lab notes on baselining a network and capturing traffic.

15.1 Suggested Next Practicals

1. Repeat Practicals One through Three while documenting each as a separate, self-contained write-up.
2. Set up a two-machine lab: one client, one server with no DNS service running, to reproduce the Chapter Seven incident from scratch.
3. Practice the port-scan detection filter from Chapter Thirteen against a machine that is owned or authorized for testing, using a tool such as `nmap`, and document the SYN-flag signature observed.
4. Combine a saved `.pcap` capture with Wireshark for a visual, annotated walkthrough.

Chapter 16

Appendix: Command and Flag Reference

16.1 Core Commands

Command	Purpose
<code>ip a</code>	List network interfaces and their IP addresses
<code>sudo tcpdump -i eth0</code>	Capture all traffic on interface eth0
<code>sudo tcpdump -i eth0 icmp</code>	Capture only ICMP traffic
<code>sudo tcpdump -i eth0 udp port 53 -nn</code>	Capture DNS traffic with raw IPs and ports
<code>sudo tcpdump -i eth0 port 80 -nn -X</code>	Capture HTTP traffic with readable payload
<code>sudo tcpdump -i eth0 'udp port 53 or icmp'</code>	Capture DNS and ICMP together
<code>sudo tcpdump -i eth0 host 8.8.8.8</code>	Capture traffic to or from a specific host
<code>sudo tcpdump -i eth0 -w capture.pcap</code>	Save a capture to a file
<code>tcpdump -r capture.pcap -nn</code>	Read back a saved capture as text
<code>dig example.com</code>	Query DNS for a domain using the default resolver
<code>dig @1.1.1.1 example.com</code>	Query DNS for a domain using a specific server
<code>ping 8.8.8.8</code>	Send ICMP echo requests to a host
<code>curl http://neverssl.com</code>	Make a plain HTTP request for testing

16.2 Flag Reference

Flag	Meaning
<code>-i <interface></code>	Interface to capture on
<code>-n</code>	Disable IP-to-hostname resolution
<code>-nn</code>	Disable IP-to-hostname and port-to-service resolution
<code>-X</code>	Show payload in hex and ASCII
<code>-x</code>	Show payload in hex only
<code>-A</code>	Show payload in ASCII only

Flag	Meaning
-w <file>	Write capture to a file
-r <file>	Read a capture from a file
-c <n>	Stop after capturing n packets
-e	Include Ethernet and MAC header information
-G <seconds> -W <count>	Rotate capture files on a timer

16.3 TCP Flag Reference

Symbol	Flag	Meaning
[S]	SYN	Connection request
[S.]	SYN-ACK	Connection accepted
[.]	ACK	Acknowledgment
[F.]	FIN-ACK	Graceful connection close
[R]	RST	Connection abruptly reset or refused

End of book.